Module Code	IY4113
Group	B
Module Title	Practical assignment part 1: Java Fundamentals
Assessment Type	Report
Module Tutor Name	Jonathan Shore
Student ID Number	P460817
Assessment Window	Final submission: 16/02/2026- 22/02/2026

☒ *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

☒ *Where I have used AI, I have cited and referenced appropriately.*

Introduction

This report presents the design, development and evaluation of the CityRide Lite application, developed as the final stage of the IY4113 module. The system is a console-based Java application that allows users to record public transport journeys, calculate fares, remove journeys and generate daily summaries.
The application enables users to add journeys using validated input, calculate fares based on predefined fare rules, apply passenger-type discounts and apply a daily cap during summary calculation. Journeys are stored in memory for the duration of the program session. The system also allows users to list all recorded journeys and remove journeys by their unique ID.
The objective of this project was to demonstrate an understanding of Java fundamentals, object-oriented programming, control structures, input validation and data handling using collections.

Analysis

The CityRide Lite system was designed to satisfy the functional requirements of the assignment. The application operates through a continuously running main menu, allowing the user to select different features until the exit option is chosen.
The system provides the following core functionality:
Add a journey
Remove a journey by ID
List all journeys
Generate a daily summary
Exit the application

Fare Calculation

Fare calculation is based on predefined zone rules within the FareCalculator class.
The base fare is determined by the selected zone:
Zone 1: £2.50
Zone 2: £3.20
Zone 3: £4.00
If the time band is Peak, an additional £0.50 is added to the base fare.
Passenger-type discounts are applied as follows:
Adult: 0%
Child: 50%
Student: 20%
Senior: 30%
The final fare is calculated by applying the relevant discount to the adjusted base fare.

Daily Cap Logic

A daily cap is applied during summary calculation. The cap is determined by the highest zone travelled on that day:
Zone 1 cap: £7.00
Zone 2 cap: £8.50
Zone 3 cap: £10.00
If the total cost of journeys for a given date exceeds the cap, the total is reduced to the cap value.

Data Management

Journeys are stored in an ArrayList within the JourneyManager class. Each journey is assigned a unique session ID, incremented automatically. Data is stored in memory only and is cleared when the application terminates.
Input validation ensures that invalid menu selections and non-existing journey IDs are handled safely. The program prevents unexpected crashes by validating user input before processing.

Input–Process–Output (IPO) Table

Add Journey

Input	Process	Output
Date entered by user	Validate date input	Valid date stored
Zone (1–3)	Validate numeric and range (1–3)	Valid zone stored
Passenger type (Adult, Child, Student, Senior)	Validate passenger type	Valid passenger type stored
Time band (Peak/OffPeak)	Validate time band	Valid time band stored
Zone	Retrieve base fare (Zone 1 £2.50, Zone 2 £3.20, Zone 3 £4.00)	Base fare
Time band	If Peak, add £0.50 surcharge	Adjusted fare
Passenger type	Apply discount (Child 50%, Student 20%, Senior 30%)	Final fare
Generated ID	Assign unique journey ID	Journey ID
All validated data	Create Journey object	Journey stored in ArrayList
Successful operation	Display confirmation message	"Journey added successfully"

Remove Journey

Input (Source)	Process	Output
User enters Journey ID	Check if ID is numeric	Error OR continue
Stored journey list	Search for matching ID	Found OR not found
Found ID	Remove journey from list	Journey removed
Updated journey list	Reset running totals	Totals cleared
Each remaining journey	Recalculate discounted fare	Updated totals
Each remaining journey	Reapply cap logic	Updated charged fares

Completed recalculation	Display success message	"Journey removed successfully"

List Journey

Input (Source)	Process	Output
User selects "List Journeys" from main menu	Validate menu selection	Continue to list function
Stored journey list (in memory)	Check if journey list is empty	If empty → "No journeys recorded" message
Stored journey list	Initialise iteration counter	Prepare loop
Each Journey object	Retrieve journey ID	
Each Journey object	Retrieve journey date	
Each Journey object	Retrieve fromZone value	
Each Journey object	Retrieve toZone value	
Each Journey object	Retrieve zonesCrossed value	
Each Journey object	Retrieve timeBand	
Each Journey object	Retrieve passengerType	
Each Journey object	Retrieve baseFare	
Each Journey object	Retrieve discountedFare	
Each Journey object	Retrieve chargedFare	
Retrieved values	Format monetary values to 2 decimal places	
Retrieved values	Format output string for display	
Completed iteration	Display all journey records in order entered	Formatted journey list

Filter Journey

Inputs	Processing	Outputs
Menu option (Filter Journeys)	Display filter menu options	Filter menu shown
Filter type selection (Passenger Type / Time Band / Zone / Date)	Validate filter selection	Error message if invalid selection
Passenger Type (if selected)	Validate passenger type against enum values	Error message if invalid
Time Band (if selected)	Validate peak/off-peak input	Error message if invalid
Zone number (if selected)	Validate zone is between 1–5	Error message if invalid
Date (if selected)	Validate date format not blank	Error message if invalid
Stored journey list (ArrayList<Journey>)	Loop through journey list	Filtered journey list
	Compare each journey's attribute with filter criteria	Matching journeys identified
	If no match found → detect empty result	"No journeys found" message
	Format output values (2 decimal places for fares)	Display filtered journeys

Daily Summary

Input	Process	Output
Date entered by user	Filter journeys by matching date	Matching journeys
Matching journeys	Sum total fares	Total cost
Matching journeys	Count total journeys	Journey count
Total cost	Determine highest zone travelled that day	Cap threshold identified
Total cost	Apply daily cap (Zone 1 £7.00, Zone 2 £8.50, Zone 3 £10.00)	Capped total
Total cost and count	Calculate average cost	Average fare
Calculated results	Display summary report	Daily summary output

Totals by passenger type

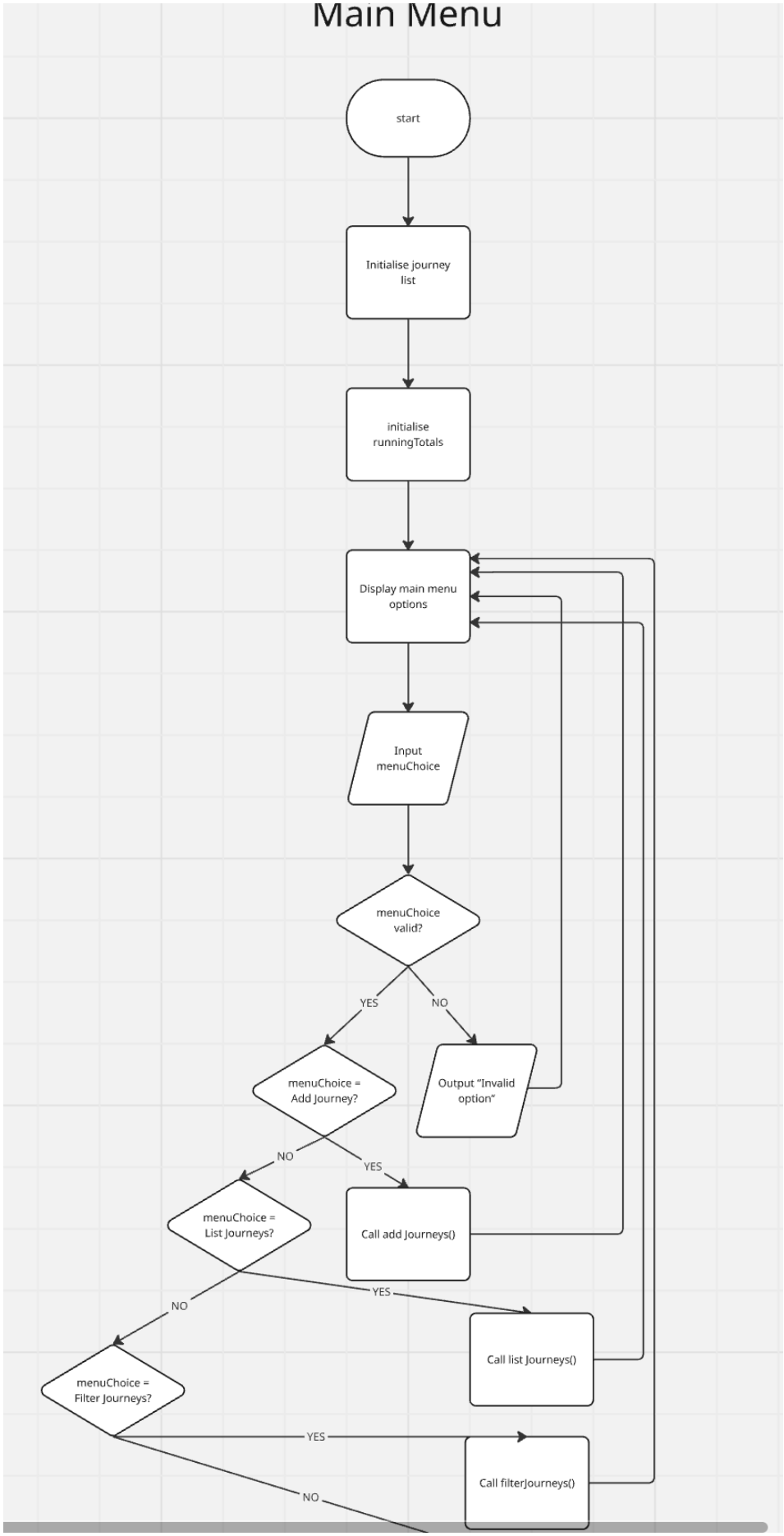
Inputs (Source)	Processing Steps	Outputs
Menu option (View Totals by Passenger Type)	Validate menu selection	Totals screen displayed
Stored journey list (ArrayList<Journey>)	Check if list is empty	"No journeys recorded" message if empty
Stored journey list	Group journeys by passenger type	Passenger groups created
Journey.passengerType	Count journeys per passenger type	Journey count
Journey.baseFare	Sum base fares per passenger type	Pre-discount total
Journey.discountedFare	Sum discounted fares per passenger type	Discounted total
Journey.chargedFare	Sum charged fares per passenger type	Charged total
Dataset DAILY_CAP value	Retrieve cap for passenger type	Cap value
Charged total	Compare charged total with cap	Cap reached (Yes/No)
Calculated totals	Format monetary values (2 decimal places)	Formatted output
All calculated values	Display results per passenger type	Structured totals report

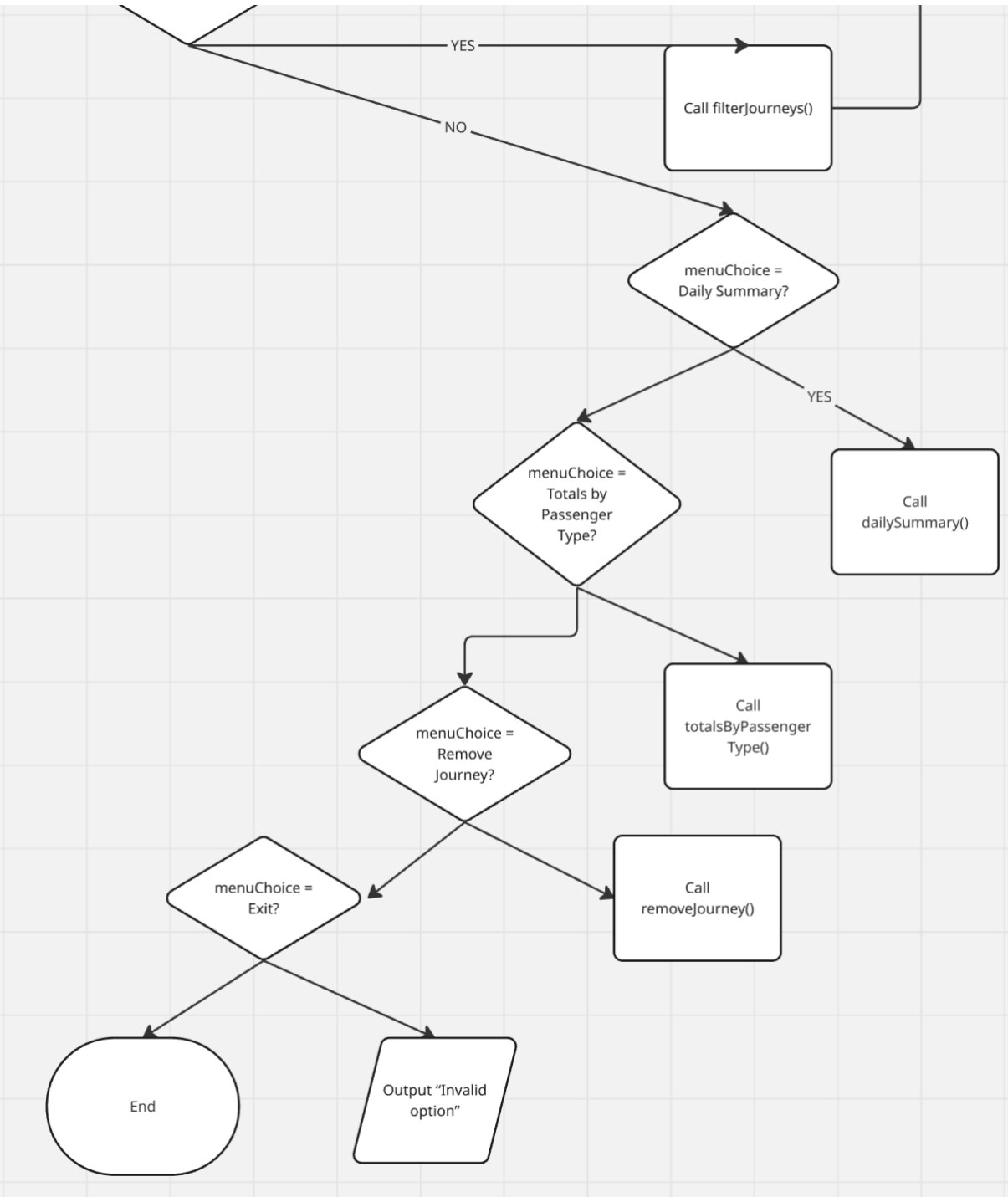
Processing Breakdown (Step-by-Step)

User selects "View Totals by Passenger Type"

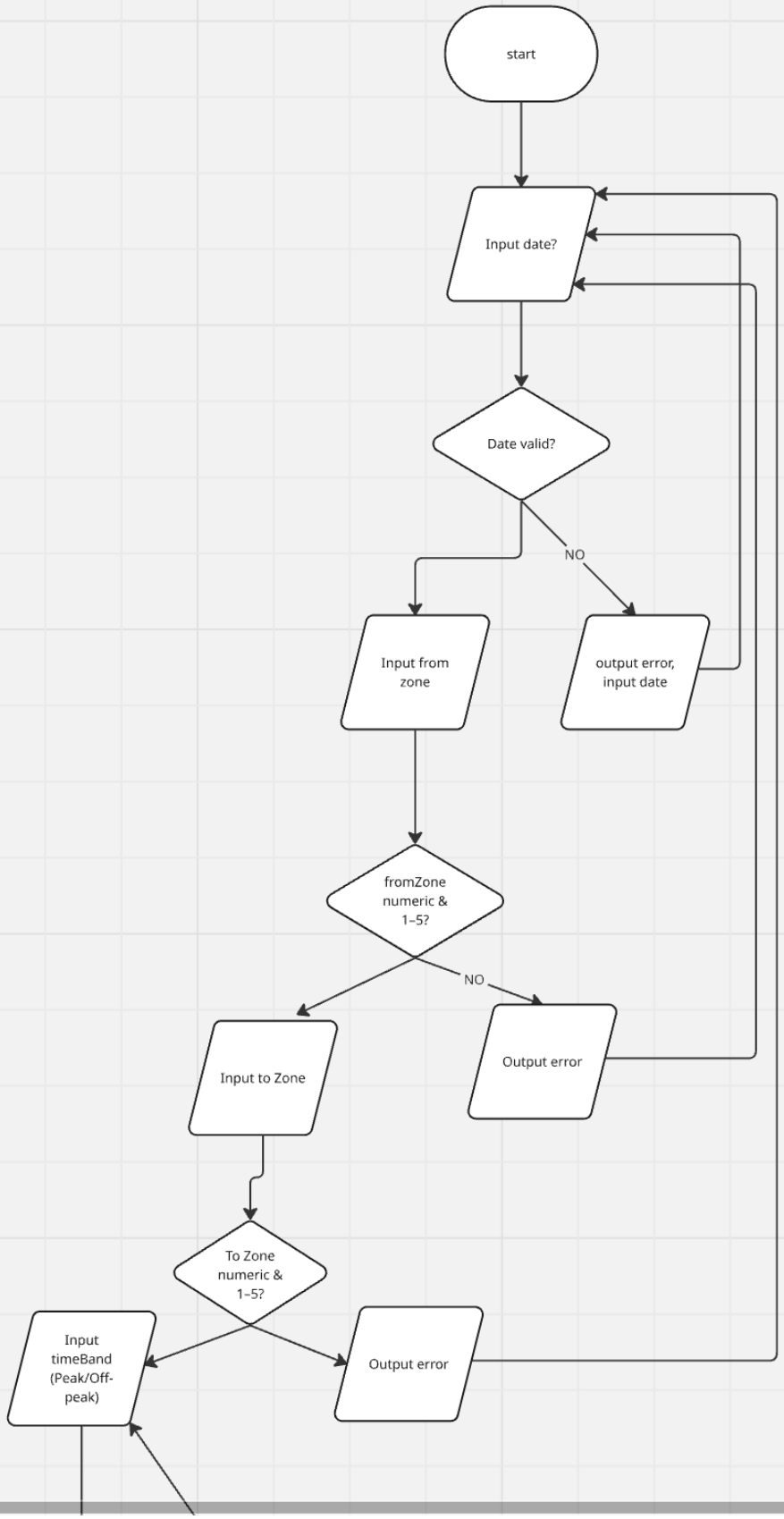
System checks if journey list is empty → If empty, display message and return

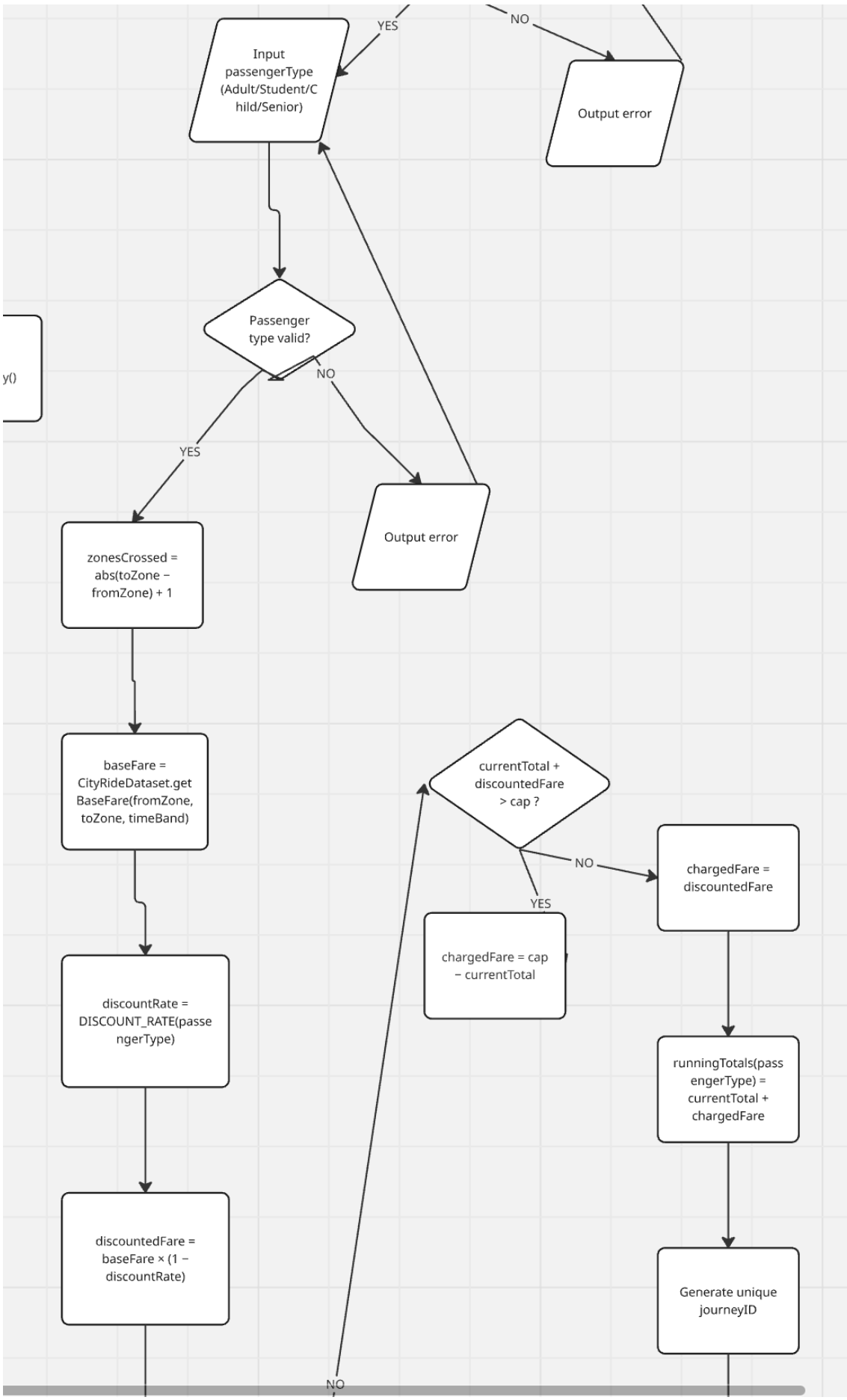
For each passenger type (Adult, Student, Child, Senior):
Count total journeys
Sum baseFare values
Sum discountedFare values
Sum chargedFare values
Retrieve daily cap from dataset
Compare charged total with cap
Determine if cap reached
Format values to 2 decimal places
Display results clearly grouped by passenger type

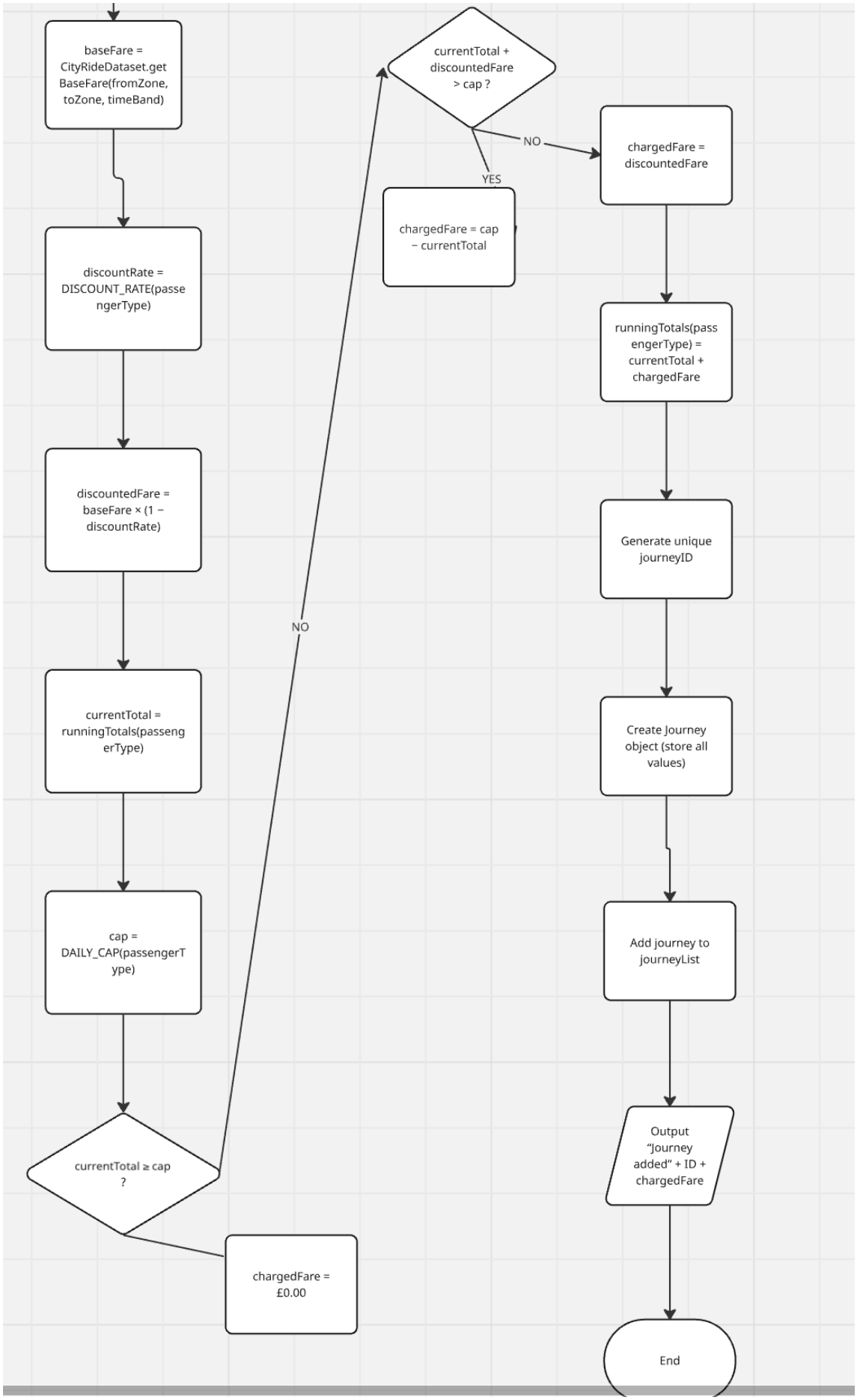

Main Menu
start
Initialise journey list
initialise runningTotals
Display main menu options
Input menuChoice
menuChoice valid?
YES
NO
menuChoice = Add Journey?
Output "Invalid option"
NO
YES
menuChoice = List Journeys?
Call add Journeys()
YES
NO
Call list Journeys()
menuChoice = Filter Journeys?
YES
Call filterJourneys()
NO

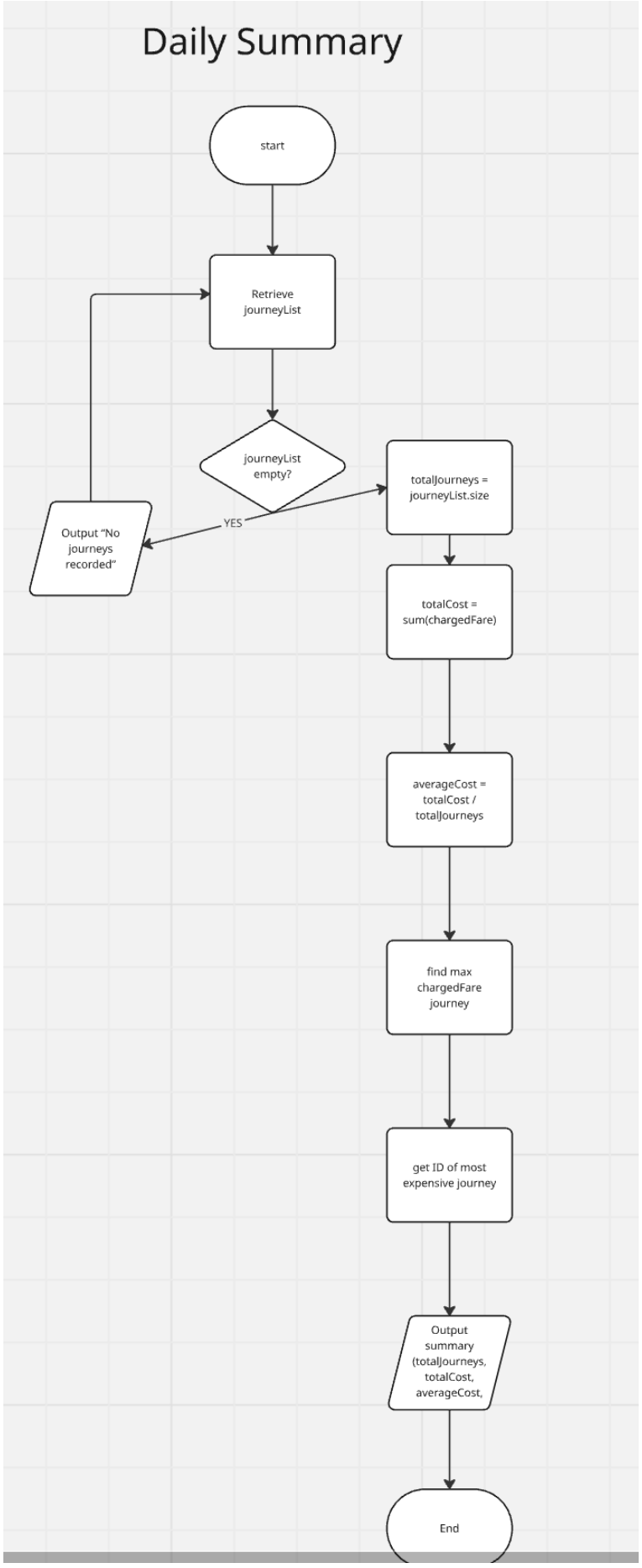

YES
Call filterJourneys()
NO
menuChoice = Daily Summary?
YES
Call dailySummary()
menuChoice = Totals by Passenger Type?
Call totalsByPassenger Type()
menuChoice = Remove Journey?
Call removeJourney()
menuChoice = Exit?
End
Output "Invalid option"

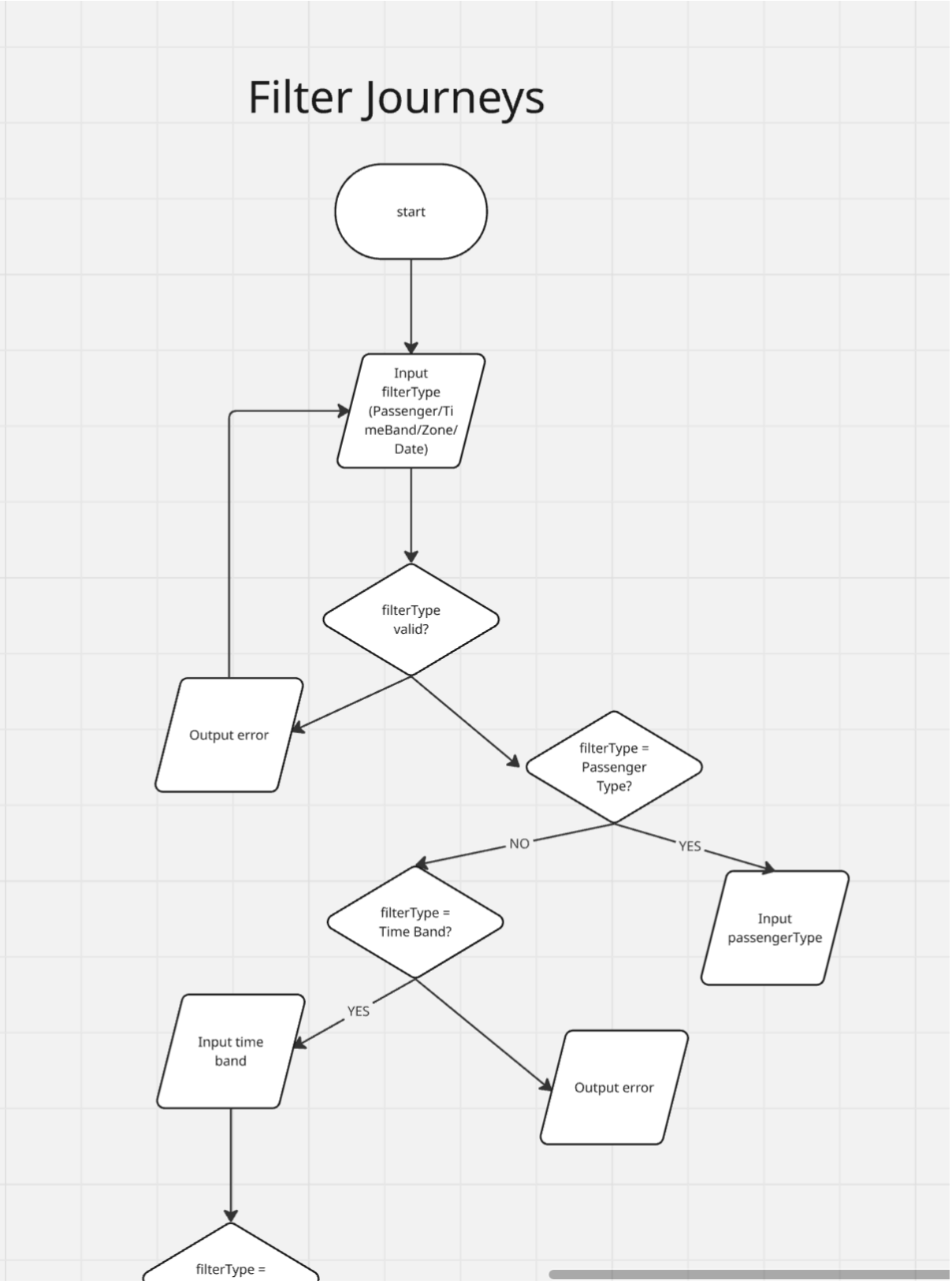
Add Journey + Fare Calculation

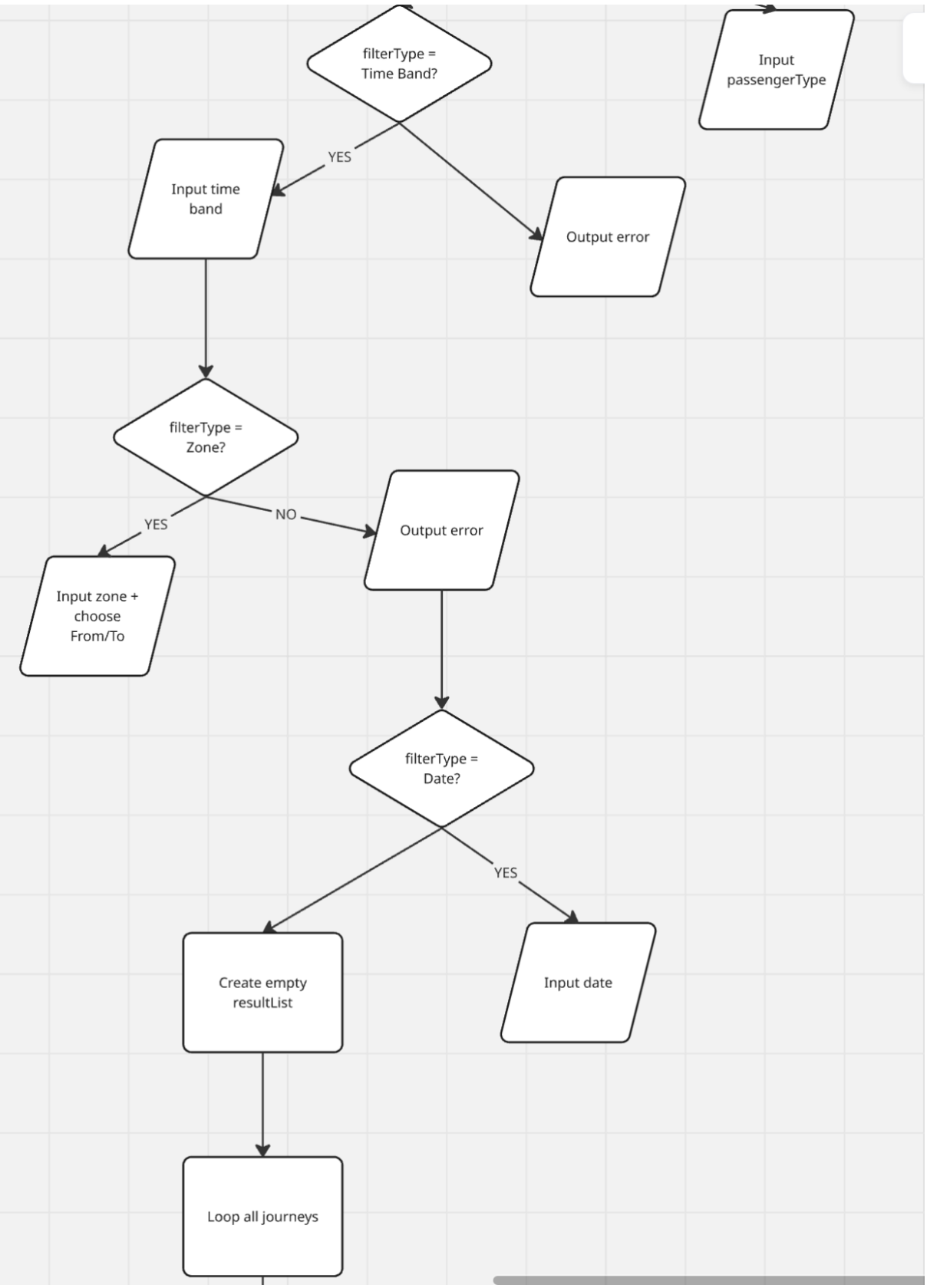


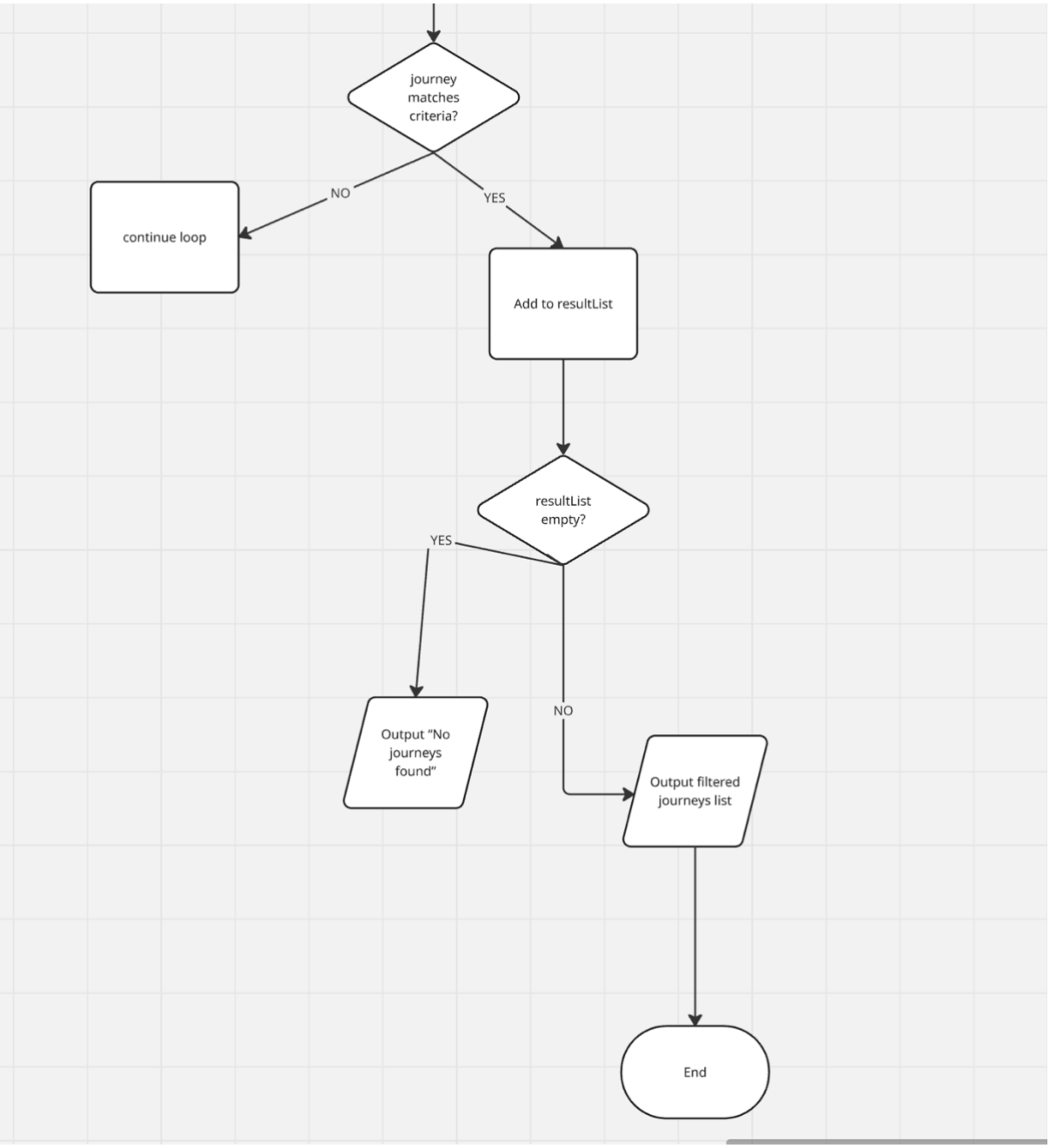

YES
NO
Input passengerType (Adult/Student/Child/Senior)
Output error
Passenger type valid?
NO
YES
Output error
zonesCrossed = abs(toZone − fromZone) + 1
baseFare = CityRideDataset.getBaseFare(fromZone, toZone, timeBand)
currentTotal + discountedFare > cap ?
NO
YES
chargedFare = discountedFare
chargedFare = cap − currentTotal
discountRate = DISCOUNT_RATE(passengerType)
runningTotals(passengerType) = currentTotal + chargedFare
discountedFare = baseFare × (1 − discountRate)
Generate unique journeyID
NO


baseFare = CityRideDataset.getBaseFare(fromZone, toZone, timeBand)
discountRate = DISCOUNT_RATE(passengerType)
discountedFare = baseFare × (1 − discountRate)
currentTotal = runningTotals(passengerType)
cap = DAILY_CAP(passengerType)
currentTotal ≥ cap ?
NO
chargedFare = £0.00
currentTotal + discountedFare > cap ?
YES
chargedFare = cap − currentTotal
NO
chargedFare = discountedFare
runningTotals(passengerType) = currentTotal + chargedFare
Generate unique journeyID
Create Journey object (store all values)
Add journey to journeyList
Output "Journey added" + ID + chargedFare
End


Daily Summary
start
Retrieve journeyList
journeyList empty?
YES
Output "No journeys recorded"
totalJourneys = journeyList.size
totalCost = sum(chargedFare)
averageCost = totalCost / totalJourneys
find max chargedFare journey
get ID of most expensive journey
Output summary (totalJourneys, totalCost, averageCost,
End


Filter Journeys
start
Input filterType (Passenger/TimeBand/Zone/Date)
filterType valid?
Output error
filterType = Passenger Type?
NO
YES
Input passengerType
filterType = Time Band?
YES
Input time band
Output error
filterType =


filterType =
Time Band?
Input
passengerType
YES
Input time
band
Output error
filterType =
Zone?
YES
NO
Output error
Input zone +
choose
From/To
filterType =
Date?
YES
Create empty
resultList
Input date
Loop all journeys


journey matches criteria?
NO
YES
continue loop
Add to resultList
resultList empty?
YES
NO
Output "No journeys found"
Output filtered journeys list
End

Testing

The system was tested using manual black-box testing. Each feature was tested using valid, invalid and boundary input values to ensure correct functionality and error handling.

Test Case ID	Feature	Input	Expected Result	Actual Result	Status
TC1	Add Journey	Date: 01/02/2026, Zone:1, Adult, Peak	Fare = £3.00, ID assigned	Fare = £3.00, ID = 1	PASS
TC2	Add Journey (Discount)	Zone:2, Child, OffPeak	Fare = £1.60	Fare = £1.60	PASS
TC3	Invalid Menu	Choice: 9	"Invalid choice!" message	Correct error displayed	PASS
TC4	Daily Summary	Date: 01/02/2026	Total=£4.60, Avg=£2.30	Correct values displayed	PASS
TC5	Remove Valid ID	ID: 1	Journey removed	Removed successfully	PASS
TC6	Remove Invalid ID	ID: 999	"Journey ID not found"	Correct message displayed	PASS
TC7	Daily Cap Test	3× Zone1 Peak Adult	Total capped at £7.00	£7.00 displayed	PASS

EVALUATION

The CityRide Lite project demonstrates a solid understanding of Java fundamentals and object-oriented programming principles. The system is structured into multiple classes, each with a clearly defined responsibility. This separation improves readability, maintainability and logical organisation.
The use of collections allows dynamic storage of journey data within a session. The unique ID mechanism ensures that journeys can be accurately identified and removed. Input validation enhances reliability by preventing invalid data from causing system failures.
Fare calculation logic correctly applies base fares, peak adjustments and passenger discounts. The daily cap mechanism ensures that the total cost does not exceed predefined thresholds, demonstrating conditional logic and control flow implementation.
However, the system has certain limitations. Data is stored only in memory and is lost when the program exits. A future improvement would be implementing file handling or database storage to allow persistent data management. Additionally, the user interface is console-based and could be enhanced with a graphical user interface for improved usability.
Further improvements could include:
Persistent file storage
Use of enumerations for stronger validation
Expanded reporting features
More advanced statistical analysis
Overall, the system successfully meets the functional requirements and demonstrates competence in Java programming, control structures, collections and modular design.

REFERENCES

Oracle (2024) *Java Platform, Standard Edition Documentation*. Available at: https://docs.oracle.com/javase/
Moovit Ltd. (2026) *Moovit Mobility Management Platform and Urban Transit Solutions*. Available at: https://moovit.com
Kaplan International Pathways. (2026). *IY4113 Practical Assignment Part 1: Java Fundamentals – CityRide Lite Specification*. Nottingham Trent University.
OpenAI. (2026). *ChatGPT (GPT-5)*. Used for structural guidance, report formatting and explanation of programming concepts. Available at: https://openai.com/

APPENDICES

Appendix A – CityRideLiteApp.java
Appendix B – Journey.java
Appendix C – FareCalculator.java
Appendix D – JourneyManager.java
Appendix E – DailySummary.java